

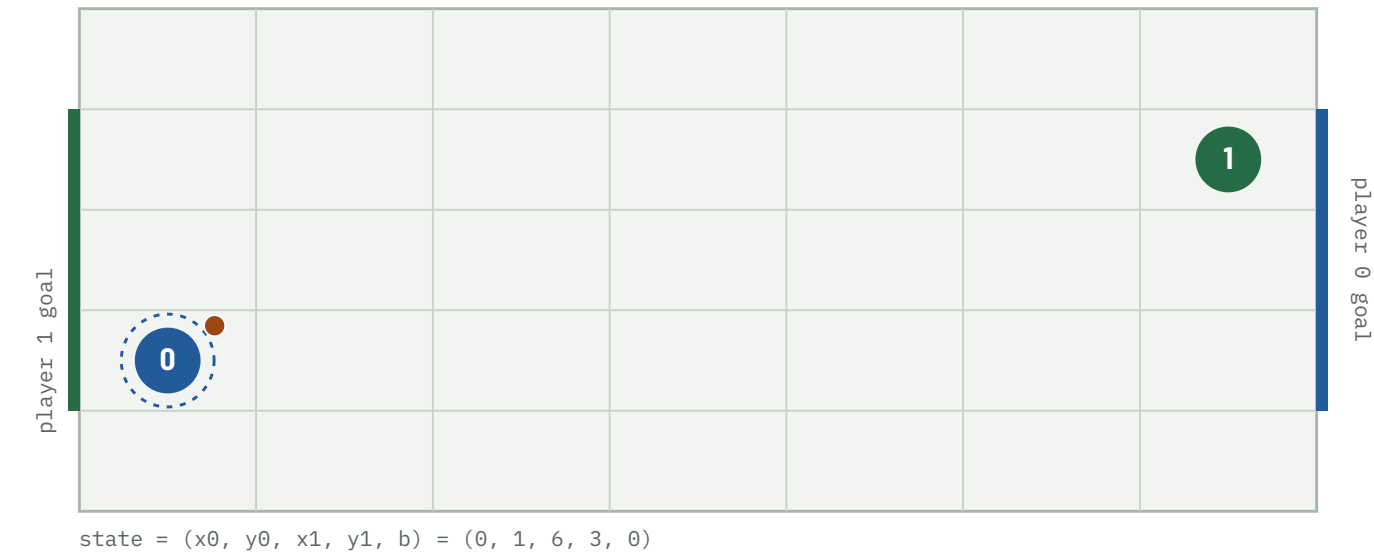
# When Does Soccer Need Mixed Strategies?

A pure-first Nash-Q approach: where  
a two-player soccer Markov game  
does — and does not — need mixing,  
and how much LP work that saves.

FOUR QUESTIONS

**RQ1 Existence** — under what transition structures does a pure equilibrium exist at every state? **RQ2 Efficiency** — how much work does checking for a pure saddle first eliminate? **RQ3 Mechanism** — which spatial configurations force mixing? **RQ4 Robustness** — how do discounting and tolerance affect the split?

repo · soccer-markov-nash | 210+ tests | game family in docs/design.md | methods in docs/methods.md | every policy drawn in docs/gallery.html



**The kickoff.** 7×5 grid, 2380 non-terminal states. Player 0 (blue) carries the ball and attacks the right edge; player 1 (green) the left. Start positions are the ones the A10 page assigns this netID. Actions U D L R are chosen simultaneously; reward is +1 / -1 / 0, zero-sum.

## ABSTRACT

- 1** The deterministic A10 game — solved exactly, undiscounted, by 100-step backward induction — has a pure saddle at every one of its 238 000 (state  $\times$  step) stage games. A pure equilibrium exists; the LP is never called; discounting was never load-bearing.

---

- 2** A coin-flip tie-break does not change this; Littman's random *move order* does — but only when the goal mouth is more than one cell wide, and even then on 3.95% of states (this configuration).

---

- 3** Those mixed states are geometrically localised: a decision tree predicts them from the state with precision 0.96, and 68 of 94 have a 2 $\times$ 2 matching-pennies support.

---

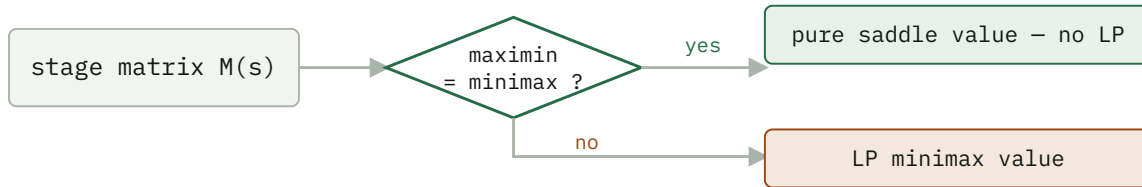
- 4** The pure-first hybrid solver matches the all-LP value function to 4e-16 while calling the LP on only 3.95% of states —  $\sim 25\times$  fewer per sweep; mirror symmetry halves the work again.

## Method

The game is zero-sum, so at every state the stage game is a payoff matrix  $M(s)[a_0, a_1] = E[R + \gamma V(s')]$  and the Shapley (1953) fixed point is the minimax value:

$$\begin{aligned} V(s) &= \text{val}(M(s)) = \max_n \min_{k \in \mathcal{K}} (p^T M)[a_1] \\ &= \min_p \max_{a_0} (Mq)[a_0] \end{aligned}$$

The  $Q = R + \beta \text{Nash}(Q')$  update is this operator. The A10 game is *undiscounted* ( $\pm 1$  at a goal, tie after 100 steps);  $\gamma < 1$  is a contraction device for value iteration and RQ4 shows the split holds across  $\gamma$ . Three stage backups: **pure** (the maximin security value — a fast lower bound, not a Nash value when it is below the minimax), **mixed** (the LP minimax value, via scipy HiGHS), and **hybrid** (the pure saddle value where maximin = minimax, the LP elsewhere — exact everywhere). Support enumeration (support\_enum.py) finds *all* equilibria, including of general-sum games.



**The pure-first hybrid.** On the deterministic game the "yes" branch is taken at every one of the 2380 states, so the LP is never called.

## RQ1 – Existence

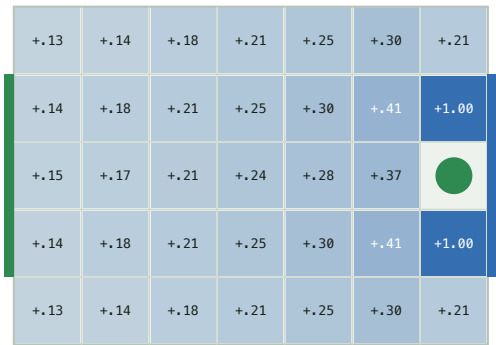
The A10 game is undiscounted, so it is solved exactly by 100-step backward induction (run\_finite\_horizon, experiments/undiscounted.csv):

EXACT UNDISCOUNTED GAME – 238 000 (STATE × STEP) STAGE GAMES

| MOVE ORDER                | MIXED STAGE GAMES  | V(KICKOFF) | PURE EQUILIBRIUM?           |
|---------------------------|--------------------|------------|-----------------------------|
| deterministic (exact A10) | 0                  | 0.000      | yes – pure (non-stationary) |
| coinflip tie-break        | 0                  | 0.000      | yes                         |
| random move order         | 3 148 (404 states) | +0.459     | no                          |

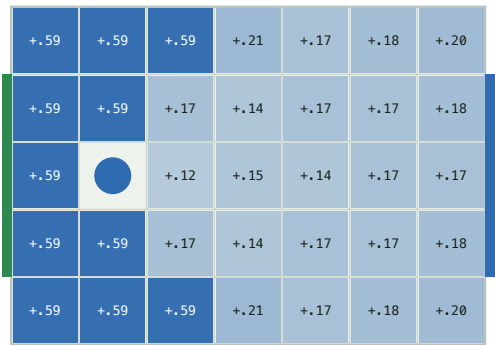
The deterministic equilibrium is **constructive and memoryless**: each player's win attractor (states from which it forces a goal, soccer\_nash/attractor.py) equals its  $V^* = \pm 1$  set, and the pure profile "attractor move on your winning set, safety move elsewhere" realizes  $V^*$  at every state (scripts/positional.py) — the game is positionally determined.

carrier position (player 0)



$V^*$  by player 0 position (blue = player 0 ahead)

defender position (player 1)



$V^*$  by player 1 position (blue = player 0 ahead)

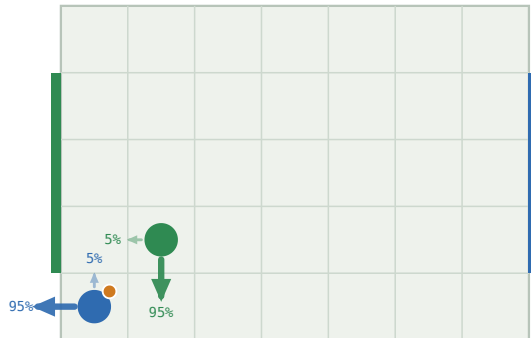
The **value surface** of the random-order game ( $\gamma = 0.9$ ). Left:  $V^*$  by carrier position – a smooth gradient toward the attacking goal, saturating to +1 on the two cells that score in one move. Right:  $V^*$  by defender position with the carrier held near midfield – nearly flat, the defender's location barely moves the value.

The stationary  $\gamma < 1$  solve agrees and is faster — 0 no-pure-saddle stage games on the deterministic game at every  $\gamma$  from 0.5 to 0.995, and 94 / 2380 on the random game at  $\gamma = 0.9$  ( $V(\text{kickoff}) = +0.164$  at the netID start). The no-pure-saddle count never depends on the kickoff.

## IT IS THE MOVE ORDER, NOT THE RANDOMNESS

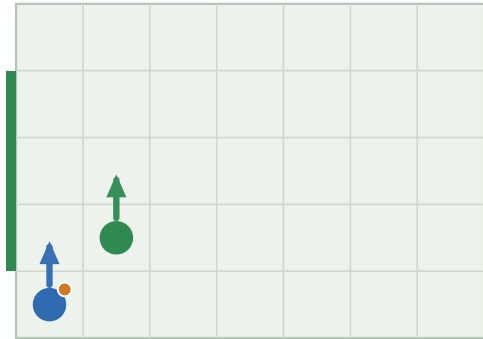
The deterministic and coin-flip value functions are *identical* ( $\max |V_{\text{det}} - V_{\text{coin}}| = 0$  at every  $\gamma$ ): optimal play never enters a losing contest, so the coin is never flipped. Only Littman's random move order — where a ball-steal depends on who resolves first *and* on both players' targets — creates matching-pennies subgames.

random order -> mixed



mixed equilibrium ·  $V = +0.078$

deterministic -> pure



pure equilibrium ·  $V = +0.000$

**One state, two resolution rules.** Arrows are the equilibrium action distribution, sized by probability. Random order (Littman) makes the stage game matching pennies; deterministic carrier-wins collapses it to a pure best reply. Full set: [docs/gallery.html](https://docs.google.com/gallery).

## RQ2 – Efficiency

Two numbers, kept apart: the **LP-call rate** is a property of the game and reproduces exactly; **wall clock** is a median of 5 repeats and depends on the machine and LP backend (scripts/benchmark.py).

RANDOM MOVE ORDER,  $\Gamma = 0.9$  – THE CASE WHERE THE LP IS ACTUALLY NEEDED

| SOLVER              | SWEEPS | LP CALLS | STATES NEEDING LP | MEDIAN WALL CLOCK |
|---------------------|--------|----------|-------------------|-------------------|
| pure (lower bound)  | 18     | 0        | –                 | 0.3 s             |
| hybrid <b>exact</b> | 108    | 9 672    | 3.95%             | 13.2 s            |
| all-LP <b>exact</b> | 108    | ~3.6 M   | 100%              | ~8 min            |

The hybrid solves an LP on only the 3.95% of states lacking a pure saddle — a  $\sim 25\times$  per-sweep reduction — and matches the all-LP value to  $4e-16$ . The wall-clock ratio ( $\sim 13$  s vs  $\sim 8$  min) is larger but less portable: it also reflects matrix construction and the LP backend. **Mirror symmetry** (run\_symmetric, deterministic dynamics only): every state has a distinct board-flip-plus-player-swap mirror, so the 2380 states are 1190 pairs — reconstruct one from the other by  $V(\text{mirror}(s)) = -V(s)$ , cutting the deterministic solve  $0.33$  s  $\rightarrow$   $0.23$  s.

## Freeze-then-iterate, and its staleness

RANDOM MOVE ORDER, HYBRID SOLVER – SAME FIXED POINT,  $|\text{DIFF}| < 3E-9$

|                  | ROUNDS / SWEEPS | MATRIX-GAME SOLVES | WALL CLOCK |
|------------------|-----------------|--------------------|------------|
| value iteration  | 108             | 9 672              | 13.8 s     |
| policy iteration | 21              | 1 879              | 17.2 s     |

$5\times$  fewer LP solves — but the frozen strategies' distance to the current Nash stays *maximal* (a full pure-strategy flip) for **16 outer rounds**, then collapses to  $< 1e-8$ . The thrashing eats the saving; on the deterministic game it is strictly worse. Value iteration with the per-sweep Nash cache wins here.

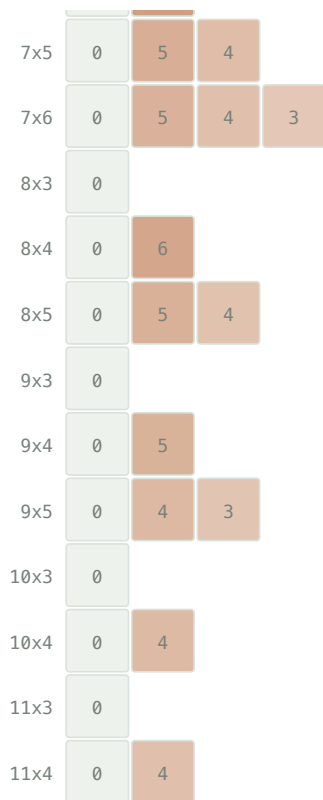
---

## RQ3 – Mechanism

**Goal-mouth width, not board size, is the gate.** The phase diagram sweeps every board with width  $\leq 11$ , height  $\leq 9$ , width $\cdot$ height  $\leq 45$  against goal widths 1 to height-2 (scripts/phase\_diagram.py --analyze, experiments/phase\_diagram.csv).

|     | goal-mouth width |    |   |   |   |   |   |
|-----|------------------|----|---|---|---|---|---|
|     | 1                | 2  | 3 | 4 | 5 | 6 | 7 |
| 3x3 | 0                |    |   |   |   |   |   |
| 3x4 | 0                | 12 |   |   |   |   |   |
| 3x5 | 0                | 10 | 9 |   |   |   |   |
| 3x6 | 0                | 7  | 7 | 7 |   |   |   |
| 3x7 | 0                | 7  | 6 | 6 | 6 |   |   |
| 3x8 | 0                | 6  | 6 | 5 | 5 | 5 |   |
| 3x9 | 0                | 6  | 5 | 5 | 4 | 4 | 4 |
| 4x3 | 0                |    |   |   |   |   |   |
| 4x4 | 0                | 8  |   |   |   |   |   |
| 4x5 | 0                | 8  | 6 |   |   |   |   |
| 4x6 | 0                | 7  | 6 | 5 |   |   |   |
| 4x7 | 0                | 7  | 5 | 5 | 4 |   |   |
| 4x8 | 0                | 6  | 5 | 5 | 4 | 4 |   |
| 4x9 | 0                | 6  | 5 | 4 | 4 | 4 | 3 |
| 5x3 | 0                |    |   |   |   |   |   |
| 5x4 | 0                | 7  |   |   |   |   |   |
| 5x5 | 0                | 6  | 4 |   |   |   |   |
| 5x6 | 0                | 6  | 4 | 4 |   |   |   |
| 5x7 | 0                | 5  | 4 | 4 | 3 |   |   |
| 5x8 | 0                | 5  | 4 | 3 | 3 | 3 |   |
| 5x9 | 0                | 5  | 4 | 3 | 3 | 3 | 3 |
| 6x3 | 0                |    |   |   |   |   |   |
| 6x4 | 0                | 8  |   |   |   |   |   |
| 6x5 | 0                | 6  | 4 |   |   |   |   |
| 6x6 | 0                | 5  | 4 | 3 |   |   |   |
| 6x7 | 0                | 4  | 4 | 4 | 3 |   |   |
| 7x3 | 0                |    |   |   |   |   |   |
| 7x4 | 0                | 7  |   |   |   |   |   |

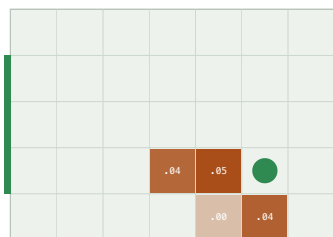




cells = mixed-state %

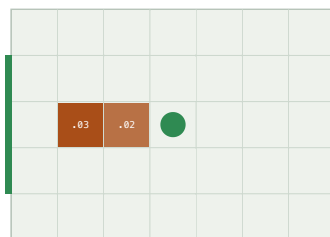
**The gate.** Goal width 1 → 0 mixed states, on all 40 one-cell configs. Goal width  $\geq 2$  → mixing on all 87: whether a board mixes is a *perfect* function of goal width. The *fraction* (2.7–12%) is an OLS  $R^2$  0.64 in board area – a dilution effect – with goal width adding little and a negative coefficient. Every state is reachable from the kickoff.

defender at (5, 1)



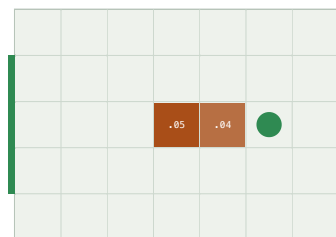
cells: minimax minus maximin of the carrier's stage game

defender at (3, 2)



cells: minimax minus maximin of the carrier's stage game

defender at (5, 2)



cells: minimax minus maximin of the carrier's stage game

**Where the carrier has to guess.** Defender pinned at the green disc; each cell shaded by minimax – maximin of the carrier's stage game (0 = pure saddle). Mixing is a local phenomenon – a handful of cells where the defender contests the forward lane.

Geometry predicts the label (scripts/geometry\_model.py): a depth-4 decision tree on carrier-frame features separates mixed from the rest with **precision 0.96, recall 0.91**. The dominant feature is defender\_can\_intercept — the defender within one move of the carrier's forward cell ( $P(\text{mixed}) = 0.44$  vs  $0.002$ ).

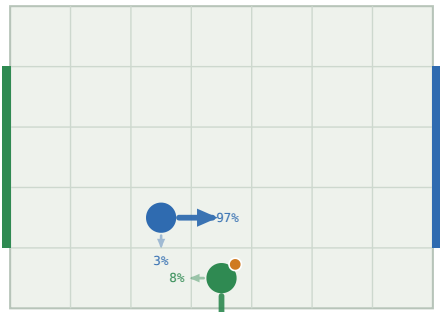
Beyond prediction, the 94 mixed states canonicalize under the board mirror to 47 pairs and cluster into 8 **geometric templates** (scripts/templates.py, docs/templates.md).

THE MIXED-STATE TEMPLATES – CARRIER-FRAME GEOMETRY, EQUILIBRIUM SUPPORT

| STATES | GEOMETRY                                               | SUPPORT | STRUCTURE             |
|--------|--------------------------------------------------------|---------|-----------------------|
| 24     | defender 1 ahead, carrier one row off the goal row     | 2×2     | matching pennies      |
| 24     | defender 1 ahead, same row, carrier on a goal row      | 2×2     | matching pennies      |
| 16     | defender 2 ahead, same row, carrier on a goal row      | 2×2     | matching pennies      |
| 4      | defender diagonally adjacent, carrier off the goal row | 2×2     | matching pennies      |
| 14     | defender 1–2 ahead, same row, on a goal row            | 2×1     | near-pure saddle      |
| 8      | defender 2 ahead, same row, on a goal row              | 3×2     | degenerate (row ties) |
| 4      | defender 2 ahead, carrier off the goal row             | 3×3     | wider mix             |

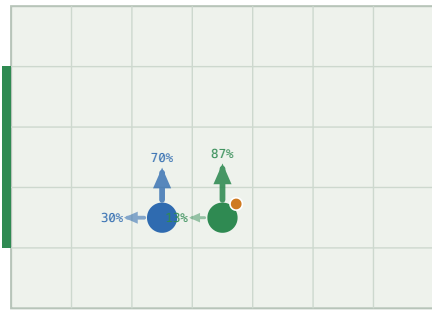
The four 2×2-support templates are all verified matching pennies — the carrier's best reply flips between the two defender columns and vice versa — covering 68 of 94 states: carrier {climb, advance} against defender {cover a lane, hold the forward cell}. All 94 share one value across every equilibrium; 64 have a unique one.

24 states  $p^*=0.92$   $q^*=0.03$



mixed equilibrium  $\cdot V = -0.150$

24 states  $p^*=0.87$   $q^*=0.70$



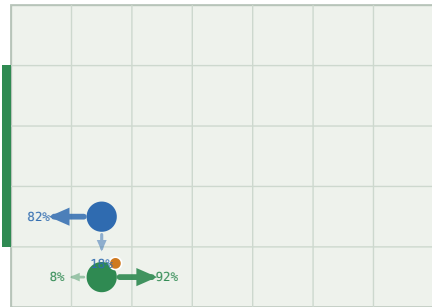
mixed equilibrium  $\cdot V = -0.199$

16 states  $p^*=0.82$   $q^*=0.66$



mixed equilibrium  $\cdot V = -0.225$

4 states  $p^*=0.08$   $q^*=0.18$



mixed equilibrium  $\cdot V = -0.206$

**One policy fan per matching-pennies template.** Each covers a class of mixed states related by carrier-frame geometry;  $p^*$ ,  $q^*$  are the equilibrium weights on the first support action. The carrier randomizes between two scoring lanes, the defender between covering them.

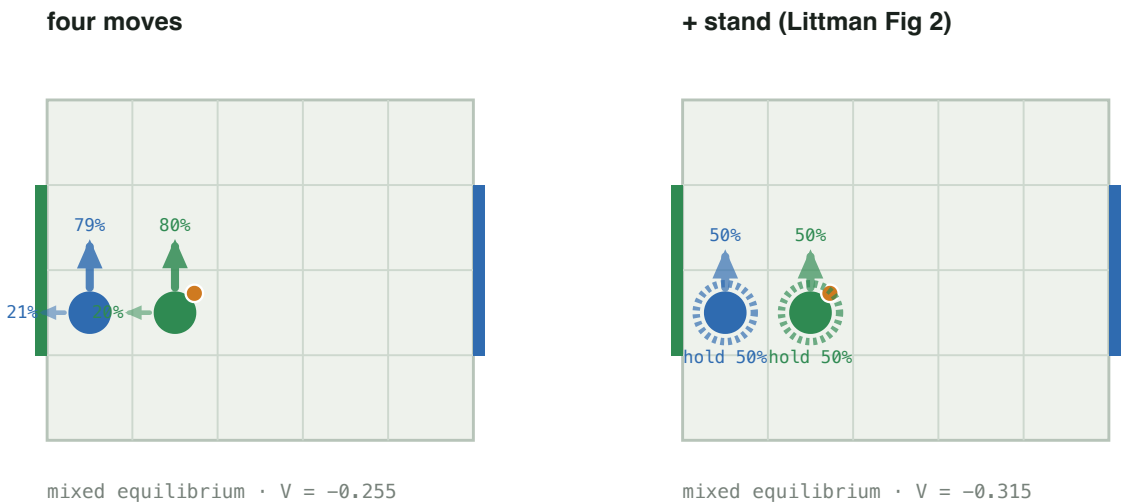
**Why the mix is unavoidable.** A mixed stage game needs all three of: a stochastic ( $\frac{1}{2}$ - $\frac{1}{2}$ ) resolution order; a goal mouth wider than one cell, so the carrier has two scoring lanes; and a defender close enough to contest the forward cell but unable to cover both lanes in one move. The carrier climbs or drops, the defender guesses which — matching pennies. Drop any one clause — a single goal cell, deterministic resolution, or the coin-flip tie-break — and every stage game has a pure saddle. The full argument and one stage matrix per template are in docs/templates.md. For the single goal cell the theorem is partly proved (docs/proof.md): the defender has a closed-form optimal strategy — guard the one cell — and every stage game is dominance-solvable, machine-checked for all boards up to  $11 \times 5$ .

## Littman's fifth action

Littman's 1994 soccer game has a fifth action, stand, and his Figure 2 — a carrier pinned against its own goal by an adjacent defender — is the textbook "soccer needs mixing" example: the carrier must randomize between standing and moving. Adding stand to the random-order game (`n_actions=5`, `scripts/littman.py`) on Littman's  $5 \times 4$  board:

| ACTION SET        | NO-SADDLE STATES | STAND IN THE MIXED SUPPORT | V(KICKOFF) |
|-------------------|------------------|----------------------------|------------|
| N, S, E, W        | 56               | 0                          | +0.147     |
| N, S, E, W, stand | 56               | 48                         | +0.172     |

The fifth action does *not* move where mixing happens — 48 of the 56 no-pure-saddle states are shared, 8 swap in and 8 out at the margin — but it changes *what* the mix is: the carrier now hedges with stand in 48 of 56 states rather than with retreat, and every player is slightly better for the option ( $V(\text{kickoff}) +0.147 \rightarrow +0.172$ ). The Figure 2 state is a clean matching-pennies matrix with equilibrium ( $\frac{1}{2}$  climb,  $\frac{1}{2}$  hold). The deterministic game keeps a pure saddle at every state with five actions too.



**Littman's Figure 2.** Same state, four moves (left) vs. five (right). The dashed ring is stand. Adding it turns the carrier's climb/retreat hedge into climb/hold — Littman's mixed policy.

## RQ4 – Numerical robustness

### The value is three numbers

`value_bracket` returns what the row player guarantees ( $\min_k (pM)_k$ ), gets ( $p^T Mq$ ), and can reach ( $\max_i (Mq)_i$ ). The true value is bracketed, so the width certifies the LP error.

FINDING

With `scipy`'s HiGHS the three agree to  $1.1e-16$  per stage game,  $8.7e-10$  accumulated. The concern is real for older vertex/simplex solvers — a non-issue here.

### Discounting and tolerance

|                                                                     | DETERMINISTIC | RANDOM (7×5)                |
|---------------------------------------------------------------------|---------------|-----------------------------|
| mixed states, $\gamma = 0.5 \rightarrow 0.9 \rightarrow 0.995$      | 0 → 0 → 0     | 122 → 94 → 102              |
| classification split, <code>rel_tol</code> $1e-12 \rightarrow 1e-3$ | stable        | 604 / 1682 / 94<br>(stable) |
| classification, <code>rel_tol</code> $1e-2 / 1e-1$                  | —             | 80 / 0 mixed                |
| fixed 0.1-rounding flips a saddle                                   | 0             | 62                          |

Discounting mildly shifts which states mix; the classification is otherwise robust across nine orders of magnitude of tolerance, breaking only as the tolerance approaches the real minimax – maximin gaps. Fixed 0.1-rounding — proposed to force indifference — misclassifies 62 states, exactly the small-entry mixed region.

---

## Secondary validation – the policy plays correctly

- Duality gap  $V0_{br}(s_0) + V1_{br}(s_0) < 1e-9$  for every move order — no opponent beats the game value.
- Nash vs. Nash reproduces the value: forced draws in the deterministic and coin-flip games;  $+0.162 \pm 0.002$  empirical discounted return over 5 seeds of 3000 games (vs.  $+0.164$  computed) in the random game.
- A best response to one assumed opponent is fragile: the Part 2 policy has exploitability  $0.43$  — worse than random — which is the A10 competition's warning about non-equilibrium submissions.
- A10 networks over 5 seeds (experiments/a10\_competition\_seeds.csv): the Part 2 imitation net plays optimally at  $0.990 \pm 0.000$  of states and wins Q8 every time; competition Network Second is exact, but Network First sits at a  $0.19 \pm 0.02$  exploitability from this kickoff that does not wash out with re-seeding.

---

## Beyond zero-sum, and open questions

- **General-sum — extension point.** `markov_game.py` solves 2-player general-sum Markov games by enumerating *all* stage equilibria (`support_enum.py`) and selecting one — the notes' "largest sum of values", a no-op for zero-sum but decisive for Battle of the Sexes, where it avoids the mixed equilibrium whose value is below either pure one. Pure-first throughout. The soccer game is zero-sum, so this does not touch the main result.
- **Function approximation — partial.** A network fit to the stage matrices (`nash_dqn.py`,  $5 \rightarrow 96 \rightarrow 96 \rightarrow 16$  with biases, 600 epochs) trails the exact solver, over 5 seeds (`experiments/nash_dqn_seeds.csv`):

|                                         | EXACT HYBRID NASH-Q | NEURAL NASH-Q (MEAN $\pm$ SD)    |
|-----------------------------------------|---------------------|----------------------------------|
| max $ V - V_{\text{exact}} $            | 0                   | 0.55 $\pm$ 0.02                  |
| mean $ V - V_{\text{exact}} $           | 0                   | 0.13 $\pm$ 0.00                  |
| action agreement                        | 100%                | 43% $\pm$ 2%                     |
| pure/mixed classification               | 100%                | 67% $\pm$ 2%                     |
| exploitability                          | $<1e-9$             | 0.44 $\pm$ 0.05                  |
| convergence (epochs to plateau, of 600) | exact               | 469 $\pm$ 31; 3/5 still creeping |
| runtime                                 | 9 sweeps, 0.3 s     | 600 epochs, ~35 s                |

Deterministic game only — `train_nash_dqn` rejects the stochastic variants, so the network never has to represent a genuinely mixed stage game. Even so it trails: it fits the ~1000 zero-value states easily (small *mean* error) but misses the  $\gamma^k$  bands and contested regions — worst-state error 0.55, policy about as exploitable as random play, mis-calls *whether* a state needs mixing a third of the time. Keep the exact solver as ground truth.

- **Four research questions** — the single-cell theorem's last step, whether the goal-width dichotomy is a known result, the move-order-vs-randomness distinction, and why freeze-then-iterate thrashes — are stated and answered as far as the evidence allows in `docs/discussion.md`.

---

## Limitations

- The A10 page redacts the ID-specific start position and goal rows; this repo uses the standard Littman geometry (configurable). Across the phase-diagram sweep every one-cell-goal board has 0 mixed states and every wider-goal board has some; the exact 3.95% is the 7×5 / 3-cell-goal case.
  - Several collision sub-cases (carrier vs. stationary opponent, bump = steal, swap possession) are *interpreted* from the two stated A10 rules, not quoted. If course staff intended otherwise, Claim A must be re-measured. See docs/assumptions.md.
  - **Claim A/B** (a pure equilibrium of the deterministic game) is established constructively — the explicit pure memoryless attractor/safety profile. **Claim C** (the theoretical game necessarily has one, over all rules and settings) is *not*, for the general goal mouth; for the single goal cell it is partly proved (docs/proof.md).
  - random and coinflip are different game definitions, isolating the collision rule and the tie-break respectively — not the A10 game.
- 

soccer\_nash — game.py · matrix\_games.py · support\_enum.py · nash\_q.py · markov\_game.py ·  
numerics.py · dominance.py · attractor.py · geometry.py · symmetry.py · tree.py ·  
templates.py · onecell.py · reachability.py · best\_response.py · exploit.py · mlp.py ·  
nash\_dqn.py · opponents.py · shaping.py · simulate.py · experiment.py · render.py · viz.py  
scripts — experiments.py · analyze.py · numerics.py · equilibria.py · geometry\_model.py ·  
templates.py · phase\_diagram.py · onecell\_proof.py · positional.py · benchmark.py ·  
policy\_iteration.py · selfplay.py · nash\_dqn.py · render.py · gallery.py · littman.py ·  
a10\_part1.py · a10\_part2.py · a10\_competition.py  
docs — design.md · methods.md · discussion.md · assumptions.md · templates.md · proof.md ·  
geometry.md · mechanism.md · littman.md · gallery.html  
experiments — baseline.csv · undiscounted.csv · gamma\_sweep.csv · tolerance\_sweep.csv ·  
board\_sweep.csv · goal\_mouth\_sweep.csv · one\_cell\_goal.csv · phase\_diagram.csv ·  
nash\_dqn\_seeds.csv  
env: 7×5 grid, 2380 states, zero-sum  $\pm 1$ ,  $\gamma = 0.9$